05-unit-specs/03/expense_tracker/api_snippets.rb
```
class API < Sinatra::Base
  def initialize(ledger:)
    @ledger = ledger
    super()
  end
end

# Later, callers do this:
app = API.new(ledger: Ledger.new)
```

This technique—passing in collaborating objects instead of hard-coding them—is known as *dependency injection* (*DI* for short). This phrase conjures up nightmares of verbose Java frameworks and incomprehensible XML files for some folks. But as the previous snippet shows, DI in Ruby is as simple as passing an argument to a method. And with it, you get several advantages:

- Explicit dependencies: they're documented right there in the signature of `initialize`
- Code that's easier to reason about (no global state)
- Libraries that are easier to drop into another project
- More testable code

One disadvantage of the way we've sketched the code here is that callers *always* have to pass in an object to record expenses. We'd like callers to be able just to say `API.new` in the common case. Fortunately, we can have our cake and eat it too. All we have to do is give the parameter a default value. Add the following code to `app/api.rb`, just inside your `API` class:

05-unit-specs/03/expense_tracker/app/api.rb
```
def initialize(ledger: Ledger.new)
  @ledger = ledger
  super()
end
```

When the HTTP POST request arrives, the `API` class will tell the `Ledger` to `record()` the expense. The return value of `record()` should indicate status and error information:

05-unit-specs/03/expense_tracker/api_snippets.rb
```
# Pseudocode for what happens inside the API class:
#
result = @ledger.record({ 'some' => 'data' })
result.success?      # => a Boolean
result.expense_id    # => a number
result.error_message # => a string or nil
```